Introdução à Computação
recursão

Alexandre Rademaker

Recursão I

Imagine queremos imprimir blocos como

```
#
##
###
####
```

Recursão II

Nosso código para isso pode ser

src/draw0.c

```c
#include <stdlib.h>
#include <stdio.h>

void draw(int n) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < i + 1; j++)
      printf("#");
    printf("\n");
  }
}

int main(int argc, char *argv[]) {
  int height = atoi(argv[1]);
  draw(height);
}
```

```
> make draw0 && ./draw0 4
#
##
###
####
```

Recursão III

Mas observe, pirâmides são estruturas recursivas!

uma pirâmide de altura 4 é na verdade uma pirâmide de altura 3 com uma fileira extra de 4 blocos adicionados.

e uma pirâmide de altura 3 é uma pirâmide de altura 2 com uma fileira extra de 3 blocos.

e uma pirâmide de altura 2 é uma pirâmide de altura 1 com uma fileira extra de 2 blocos.

e, finalmente, uma pirâmide de altura 1 é uma pirâmide de altura 0 (sem blocos) com uma linha de 1 bloco adicionada.

Recursão IV

src/draw1.c

```c
#include <stdlib.h>
#include <stdio.h>

void draw(int n) {
  if (n <= 0)  return;
  draw(n - 1);
  for (int i = 0; i < n; i++)
    printf("#");
  printf("\n");
}

int main(int argc, char *argv[]) {
  int height = atoi(argv[1]);
  draw(height);
  return 0;
}
```

Recursão eficiente I

src/math.c

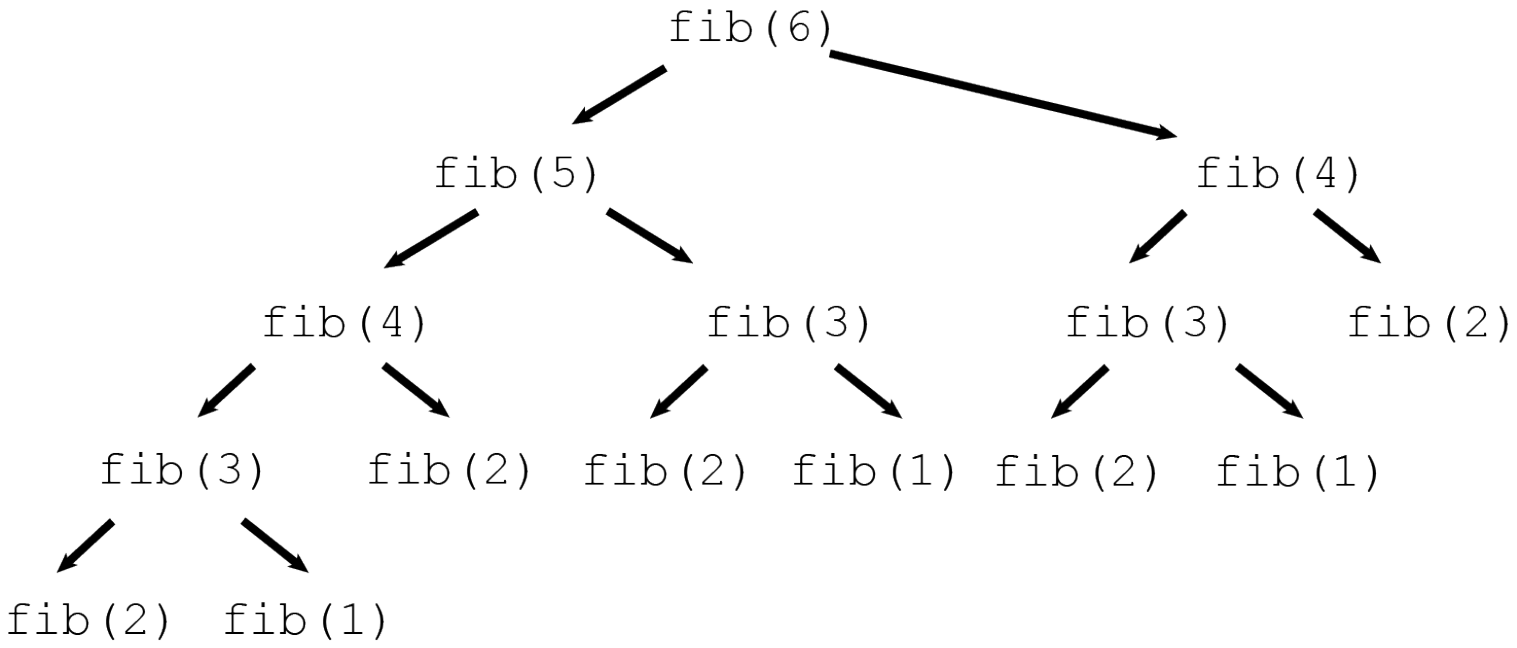
```
1 long int fib (int n) {
2   if( n == 1 || n == 2)
3     return 1;
4   else
5     return fib(n - 1) + fib(n - 2);
6 }
```

Qual o problema com esta solução para Fibonacci:

$$F_n = F_{n-1} + F_{n-2}$$

onde $F_1 = F_2 = 1$?

Recursão eficiente II



Repetição desnecessária de computações. Como corrigir?

Recursão eficiente III

$$fib_{aux}(4,1,1) = fib_{aux}(3,1,2)$$
$$fib_{aux}(3,1,2) = fib_{aux}(2,2,3)$$
$$fib_{aux}(2,2,3) = fib_{aux}(1,3,5)$$
$$fib_{aux}(1,3,5) = 3$$

src/math.c

```c
long int fib_aux(int n, int a, int b) {
  return (--n > 0) ? fib_aux(n, b, a + b) : a ;
}

long int fib(int n) {
  return fib_aux(n, 1, 1);
}
```

Busca Binária I

Podemos agora repensar na função de busca binária!

Considerando um array ordenado, como recursivamente verificar se um elemento N está no array?

A cada passo, queremos buscar em um novo array, alguma metade do array inicial.

Busca Binária II

src/numbers3.c

```
1  int bin_search_aux(int a[], int x, int i, int j) {
2    if (j < i) return -1;
3
4    int k = (j + i) / 2;
5    if (a[k] == x)
6     return k;
7    else if (a[k] < x)
8     return bin_search_aux(a, x, k + 1, j);
9    else
10    return bin_search_aux(a, x, i, k - 1);
11 }
12
13 int bin_search(int a[], int x, int n) {
14   return bin_search_aux(a, x, 0, n - 1);
15 }
```

Como **debugar**? Com alguns **printf**...

Busca Binária III

De uma galáxia muito muito distante...

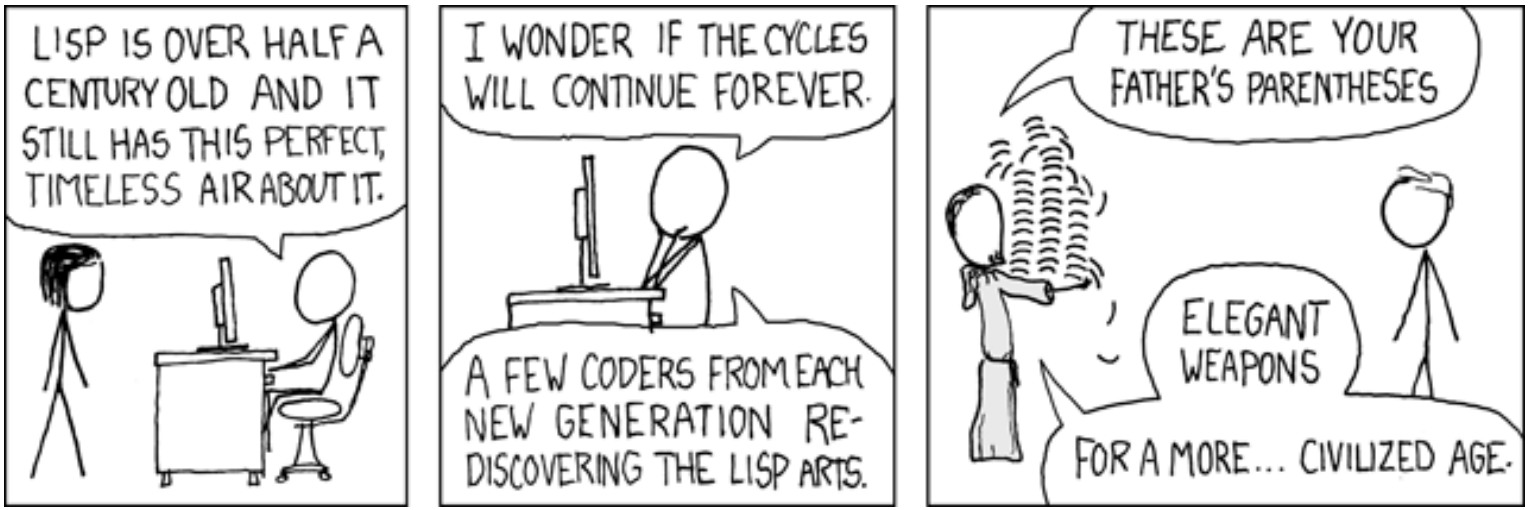
```
(defun binary-search (value array &optional (low 0) (high (1- (length array))))
    (if (< high low)
        nil
        (let ((middle (floor (+ low high) 2)))
          (cond ((> (aref array middle) value)
                 (binary-search value array low (1- middle)))
                ((< (aref array middle) value)
                 (binary-search value array (1+ middle) high))
                (t middle)))))

```

```
CL-USER> (trace binary-search)
  (BINARY-SEARCH)

CL-USER> (binary-search 4 #(-31 0 1 2 2 4 65 83 99 782))
 0: (BINARY-SEARCH 4 #(-31 0 1 2 2 4 65 83 99 782))
  1: (BINARY-SEARCH 4 #(-31 0 1 2 2 4 65 83 99 782) 5 9)
   2: (BINARY-SEARCH 4 #(-31 0 1 2 2 4 65 83 99 782) 5 6)
   2: BINARY-SEARCH returned 5
  1: BINARY-SEARCH returned 5
 0: BINARY-SEARCH returned 5
 5
```

Busca Binária IV



Fonte https://xkcd.com/297/